UNIVERSIDAD DE LA AMAZONIA
Facultad de Ingeniería
Programa de Ingeniería de Sistemas Logica & Algoritmos II

GUÍA PRÁCTICA

1. Presentación y Objetivos Pedagogicos

En la asignatura Lógica y Algoritmos II de la Universidad de la Amazonia, la transición desde estructuras fijas hacia colecciones dinámicas representa un salto cuantitativo y cualitativo en la madurez del estudiante de Ingeniería de Sistemas. El presente taller plantea **cinco retos escalables** basados en el caso de estudio de la Comercializadora Agroecológica Amazónica.

- **Reto 1 (Captura Interactiva):** Entrada de datos por consola con Scanner e instanciación repetitiva con .add().
- **Reto 2 (Búsqueda y Catálogo):** Algoritmo de búsqueda lineal por código mediante comparaciones String.equals().
- **Reto 3 (Módulo CRM de Clientes):** Control de duplicados por cédula y filtrado de sublistas dinámicas de facturas.
- **Reto 4 (Métricas e Indicadores):** Búsqueda de máximos/mínimos y acumuladores condicionales sobre colecciones.
- **Reto 5 (Eliminación de Ítems):** Manipulación física con .remove(index) y recalculo automático del estado del objeto.

2. Reto 1: Captura Interactiva de Productos por Consola

Enunciado: El punto de venta requiere ingresar en tiempo real los nuevos productos regionales recibidos (arazá, copoazú, queso caqueteño). Diseñe un método en una clase de prueba que solicite por consola (java.util.Scanner) los datos de cada producto y los agregue dinámicamente a un ArrayList<Producto>.

```
// --- SOLUCIÓN RETO 1: CAPTURA INTERACTIVA POR CONSOLA ---
package co.edu.uniamazonia.tiendaamazonica.prueba;

import co.edu.uniamazonia.tiendaamazonica.mundo.Producto;
import java.util.ArrayList;
import java.util.Scanner;

public class Reto1CapturaProductos {

    public static ArrayList<Producto> capturarProductosConsola() {
        Scanner scanner = new Scanner(System.in);
        ArrayList<Producto> listaCapturada = new ArrayList<>();
        String continuar = "S";

        System.out.println("=== REGISTRO INTERACTIVO DE PRODUCTOS ===");
        while (continuar.equalsIgnoreCase("S")) {
            System.out.print("Ingrese código del producto (ej. P001): ");
            String codigo = scanner.nextLine();
```

```
            System.out.print("Ingrese nombre del producto: ");
            String nombre = scanner.nextLine();

            System.out.print("Ingrese precio unitario ($): ");
            double precio = Double.parseDouble(scanner.nextLine());

            // Instanciación del objeto y adición dinámica
            Producto p = new Producto(codigo, nombre, precio);
            listaCapturada.add(p);

            System.out.print("¿Desea registrar otro producto? (S/N): ");
            continuar = scanner.nextLine();
            System.out.println("-----------------------------------");
        }
        System.out.println("-> Registro finalizado. Total productos ingresados: " +
listaCapturada.size());
        return listaCapturada;
    }
}
```

3. Reto 2: Gestor de Catálogo y Búsqueda por Código

Enunciado: Diseñe la clase CatalogoProductos que administre private ArrayList<Producto> productos;. Implemente el método buscarProductoPorCodigo(String codigo) que recorra la colección con un ciclo for y compare los códigos con .equalsIgnoreCase(). Si lo encuentra, retorna el objeto; de lo contrario, retorna null.

```
// --- SOLUCIÓN RETO 2: CLASE CATALOGOPRODUCTOS ---
package co.edu.uniamazonia.tiendaamazonica.mundo;

import java.util.ArrayList;

public class CatalogoProductos {

    private ArrayList<Producto> productos;

    public CatalogoProductos() {
        // Instanciación obligatoria en memoria
        this.productos = new ArrayList<>();
    }

    public void agregarProducto(Producto p) {
        if (p != null) {
            this.productos.add(p);
        }
    }

    /**
```

```
     * Búsqueda lineal en el ArrayList comparando cadenas de texto.
     */
    public Producto buscarProductoPorCodigo(String codigoBusqueda) {
        if (codigoBusqueda == null) return null;

        for (int i = 0; i < this.productos.size(); i++) {
            Producto p = this.productos.get(i);
            if (p.darCodigo().equalsIgnoreCase(codigoBusqueda)) {
                return p; // Referencia encontrada
            }
        }
        return null; // No existe en la colección
    }

    public int darCantidadProductos() {
        return this.productos.size();
    }
}
```

4. Reto 3: Módulo CRM y Registro de Clientes

Enunciado: Cree la clase GestionClientes (CRM) que administre ArrayList<Cliente> y ArrayList<Factura>. Implemente registrarCliente() asegurando que la cédula no esté duplicada antes de guardar, y el método consultarFacturasPorCliente(String cedula) que retorne una nueva lista filtrada con las facturas del usuario.

```
// --- SOLUCIÓN RETO 3: CLASE GESTIONCLIENTES (CRM) ---
package co.edu.uniamazonia.tiendaamazonica.mundo;

import java.util.ArrayList;

public class GestionClientes {

    private ArrayList<Cliente> listaClientes;
    private ArrayList<Factura> listaFacturas;

    public GestionClientes() {
        this.listaClientes = new ArrayList<>();
        this.listaFacturas = new ArrayList<>();
    }

    public boolean registrarCliente(Cliente nuevoCliente) {
        if (nuevoCliente == null) return false;

        // Validar no duplicación de cédula
        for (int i = 0; i < this.listaClientes.size(); i++) {
            if
(this.listaClientes.get(i).darCedula().equalsIgnoreCase(nuevoCliente.darCedula())) {
                System.out.println("Error: La cédula " + nuevoCliente.darCedula() + " ya
```

```
está registrada.");
                return false;
            }
        }
        this.listaClientes.add(nuevoCliente);
        return true;
    }

    public void registrarFactura(Factura f) {
        if (f != null) {
            this.listaFacturas.add(f);
        }
    }

    /**
     * Retorna una nueva lista filtrada conteniendo solo las facturas del cliente.
     */
    public ArrayList<Factura> consultarFacturasPorCliente(String cedulaCliente) {
        ArrayList<Factura> facturasEncontradas = new ArrayList<>();

        for (int i = 0; i < this.listaFacturas.size(); i++) {
            Factura f = this.listaFacturas.get(i);
            if (f.darCliente() != null &&
f.darCliente().darCedula().equalsIgnoreCase(cedulaCliente)) {
                facturasEncontradas.add(f);
            }
        }
        return facturasEncontradas;
    }
}
```

5. Reto 4: Reportes e Indicadores Financieros

Enunciado: Implemente métodos de análisis financiero sobre las colecciones: obtenerProductoMasCostoso() para encontrar el precio máximo en el catálogo, calcularVentasTotalesTienda() para acumular las ventas globales, y contarFacturasSuperioresA(double monto) para auditoría.

```
// --- SOLUCIÓN RETO 4: MÉTODOS DE REPORTES FINANCIEROS ---
package co.edu.uniamazonia.tiendaamazonica.mundo;

import java.util.ArrayList;

public class ReportesFinancieros {

    public static Producto obtenerProductoMasCostoso(ArrayList<Producto> catalogo) {
        if (catalogo == null || catalogo.isEmpty()) return null;

        Producto masCostoso = catalogo.get(0);
```

```
        for (int i = 1; i < catalogo.size(); i++) {
            Producto actual = catalogo.get(i);
            if (actual.darPrecioUnitario() > masCostoso.darPrecioUnitario()) {
                masCostoso = actual;
            }
        }
        return masCostoso;
    }

    public static double calcularVentasTotalesTienda(ArrayList<Factura> facturas) {
        if (facturas == null) return 0.0;

        double totalGlobal = 0.0;
        for (int i = 0; i < facturas.size(); i++) {
            totalGlobal += facturas.get(i).calcularTotal();
        }
        return totalGlobal;
    }

    public static int contarFacturasSuperioresA(ArrayList<Factura> facturas, double
montoMinimo) {
        if (facturas == null) return 0;

        int contador = 0;
        for (int i = 0; i < facturas.size(); i++) {
            if (facturas.get(i).calcularTotal() > montoMinimo) {
                contador++;
            }
        }
        return contador;
    }
}
```

6. Reto 5: Eliminación Física de Ítems en Factura

Enunciado: Añada a la clase Factura el método eliminarDetallePorCodigoProducto(String codigoProducto). Debe localizar la posición del renglón cuyo producto coincida con el código, eliminarlo físicamente con .remove(index) y actualizar de inmediato el total de la factura.

```
// --- SOLUCIÓN RETO 5: MÉTODO DE ELIMINACIÓN EN FACTURA.JAVA ---
// (Agregar este método dentro de la clase Factura)

public boolean eliminarDetallePorCodigoProducto(String codigoProducto) {
    if (codigoProducto == null || this.detalles.isEmpty()) return false;

    for (int i = 0; i < this.detalles.size(); i++) {
        DetalleFactura det = this.detalles.get(i);
        if (det.darProducto() != null &&
det.darProducto().darCodigo().equalsIgnoreCase(codigoProducto)) {
```

```
            // Eliminación física del elemento en memoria
            this.detalles.remove(i);

            // Recalcular el total acumulado de la factura
            this.calcularTotal();
            System.out.println("-> Producto " + codigoProducto + " retirado con éxito de la
factura.");
            return true;
        }
    }
    System.out.println("-> No se encontró el producto " + codigoProducto + " en el detalle
de la factura.");
    return false;
}
```

7. Clase Principal de Demostración e Integración

A continuación se presenta la clase ejecutable de prueba que integra y valida la respuesta a los 5 retos en Apache NetBeans:

```
// --- CLASE DEMOSTRATIVA DE PRUEBA INTEGRADORA ---
package co.edu.uniamazonia.tiendaamazonica.prueba;

import co.edu.uniamazonia.tiendaamazonica.mundo.*;
import java.util.ArrayList;

public class DemostraciónRetosTienda {

    public static void main(String[] args) {
        System.out.println("================================================");
        System.out.println("  TIENDA AMAZÓNICA - DEMOSTRACIÓN DE RETOS 1 A 5  ");
        System.out.println("================================================\n");

        // --- RETO 2: CATÁLOGO DE PRODUCTOS ---
        CatalogoProductos catalogo = new CatalogoProductos();
        Producto p1 = new Producto("P001", "Pulpa de Arazá 500g", 8500.0);
        Producto p2 = new Producto("P002", "Queso Caqueteño 1kg", 24000.0);
        Producto p3 = new Producto("P003", "Mermelada de Copoazú", 12000.0);

        catalogo.agregarProducto(p1);
        catalogo.agregarProducto(p2);
        catalogo.agregarProducto(p3);

        System.out.println("[RETO 2] Búsqueda de P002: " +
catalogo.buscarProductoPorCodigo("P002").darNombre());

        // --- RETO 3: CRM Y CLIENTES ---
        GestionClientes crm = new GestionClientes();
        Cliente c1 = new Cliente("11223344", "María Caicedo",
```

```
"m.caicedo@uniamazonia.edu.co");
        crm.registrarCliente(c1);

        Factura f1 = new Factura(1001, "2026-09-22", c1);
        f1.agregarDetalle(new DetalleFactura(3, p1)); // 3 * 8500 = 25500
        f1.agregarDetalle(new DetalleFactura(2, p2)); // 2 * 24000 = 48000
        f1.calcularTotal(); // Total: 73500

        crm.registrarFactura(f1);

        ArrayList<Factura> facturasMaria = crm.consultarFacturasPorCliente("11223344");
        System.out.println("[RETO 3] Facturas de María: " + facturasMaria.size() + "
encontrada(s).");

        // --- RETO 4: REPORTES FINANCIEROS ---
        ArrayList<Producto> listaProds = new ArrayList<>();
        listaProds.add(p1); listaProds.add(p2); listaProds.add(p3);

        Producto masCaro = ReportesFinancieros.obtenerProductoMasCostoso(listaProds);
        System.out.println("[RETO 4] Producto más costoso: " + masCaro.darNombre() + " ($" +
masCaro.darPrecioUnitario() + ")");

        ArrayList<Factura> listaFacturasGlobal = new ArrayList<>();
        listaFacturasGlobal.add(f1);
        double ventas =
ReportesFinancieros.calcularVentasTotalesTienda(listaFacturasGlobal);
        System.out.println("[RETO 4] Ventas totales globales: $" + ventas);

        // --- RETO 5: ELIMINACIÓN DE ÍTEMS ---
        System.out.println("\n[RETO 5] Total antes de eliminar: $" + f1.calcularTotal());
        f1.eliminarDetallePorCodigoProducto("P001"); // Elimina la pulpa de arazá
        System.out.println("[RETO 5] Total después de eliminar: $" + f1.calcularTotal());

        System.out.println("\n================================================");
        System.out.println("     PRUEBA FINALIZADA CON ÉXITO EN NETBEANS      ");
        System.out.println("================================================");
    }
}
```

8. Lista de Chequeo y Autoevaluación del Estudiante

Utilice la siguiente tabla de auditoría técnica antes de entregar la solución de los retos:

Criterio	Descripción del Requisito Técnico	Estado ([X])
Reto 1	Maneja la lectura con Scanner y agrega objetos al ArrayList dinámicamente sin tamaño fijo.	[ ] Cumplido
Reto 2	Implementa búsqueda lineal con .size() y compara cadenas usando .equalsIgnoreCase().	[ ] Cumplido
Reto 3	Valida que no existan cédulas duplicadas y genera sublistas filtradas de facturas.	[ ] Cumplido
Reto 4	Calcula correctamente extremos (máximos) y acumulados sobre colecciones dinámicas.	[ ] Cumplido
Reto 5	Aplica .remove(index) para eliminar elementos físicos y actualiza el saldo del objeto.	[ ] Cumplido